01 QUESTION ONE · COMPONENT CLUSTERING

How clustering factor improves and helps in COM programming?

4 A4 PAGES
C++ PROGRAM ON PAGE 4

INTRODUCTION TO COM

COM (Component Object Model) is a Microsoft specification, introduced in **1993**, that describes how software components should be built and how they should talk to one another. A *component* is a ready-made piece of software that performs one clear job and can be plugged into different programs without being recompiled.

Before COM, reusable code was shared through ordinary DLLs, but that method created serious trouble â€" there was no fixed way to ask a DLL what it could do, C and C++ compilers mangled function names differently, and installing a new DLL often broke an older program. This problem was so common that it was nicknamed **â€œDLL Hellâ€**.

COM solved this by defining a **binary standard**. A component written in C++, Visual Basic or Delphi can be used by a program written in a completely different language, because they agree only on the layout of a virtual table and not on the source code.



FIG. 1 — THE COM CALL COST LADDER. A HIGHER CLUSTERING FACTOR KEEPS THE CHATTY OBJECTS ON THE LEFT, WHERE EACH CALL IS CHEAPEST.

WHAT IS THE CLUSTERING FACTOR?

Clustering means keeping related things together so that they can work as one group. In COM the **clustering factor** measures how well the objects that talk most often have been kept together, in the same process or at least on the same machine, so that their calls stay **local**. The cost of a call is not the same everywhere. A call inside one process is a direct jump through a v-table and takes nanoseconds. A call crossing an **apartment** boundary needs **marshalling** â€” a proxy packs the arguments, a stub unpacks them, the call runs and the result is packed back. A call leaving the process adds context switches, and for DCOM a full network round-trip. The cost therefore grows from nanoseconds to microseconds to milliseconds as we move outward.

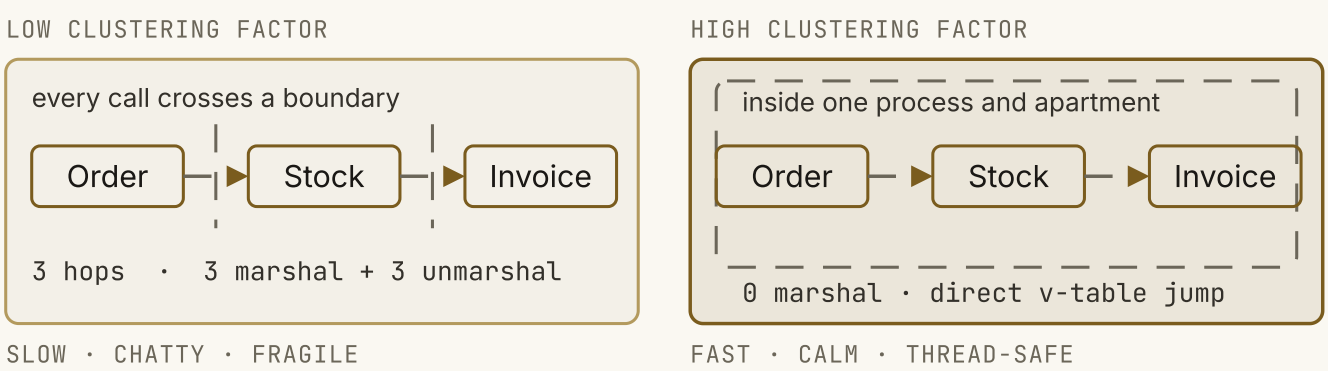


FIG. 2 — THE SAME THREE OBJECTS, ARRANGED TWO WAYS. ON THE LEFT EVERY MESSAGE CROSSES A BOUNDARY AND MUST BE MARSHALLED; ON THE RIGHT THE FREQUENTLY-TALKING OBJECTS LIVE INSIDE ONE BOUNDARY, SO THE CALLS TURN INTO ORDINARY LOCAL JUMPS.

HOW THE CLUSTERING FACTOR IMPROVES COM

A good clustering factor improves a COM system in four clear ways.

First, it removes marshalling overhead. When two objects sit inside one process, no proxy and no stub have to be built, so the arguments are passed straight through the v-table. Both CPU cost and call delay then fall sharply.

Second, it improves throughput. A business object that calls a data object a thousand times in a single transaction becomes noticeably faster when the two are clustered, because a thousand marshalled calls become a thousand direct calls.

Third, it improves cache locality. The code and data of related objects stay close together in memory, so the processor finds them without going out to the main memory.

Fourth, it simplifies apartment and thread management and scales reliably, because the objects that must not share a thread are exactly the ones that were never put across a boundary.

POINT OF COMPARISON	LOW FACTOR	HIGH FACTOR
Call path	Cross-process / cross-machine	In-process (direct v-table)
Marshalling cost	High â€” packed every call	Almost zero
Latency per call	Microseconds to milliseconds	A few nanoseconds

To raise the clustering factor in practice, an application should declare every set of components that are always used together inside the **same apartment**, should avoid needless interfaces between objects of the same tier, and should think carefully before splitting a program across the network. A bank, for instance, keeps its account, ledger and validation objects together in one clustered tier, so a transaction hits the fast local path many times.

C++ PROGRAM

CLUSTER_DEMO.CPP

```cpp
// cluster_demo.cpp - related components, local calls
#include <iostream>
#include <string>
using namespace std;
class IComponent {                        // COM-style interface
public: virtual void run(string t) = 0; };
class Account : public IComponent {       // member 1
public: void run(string t){ cout<<"[Acc] "<<t<<endl; } };
class Ledger : public IComponent {        // member 2
public: void run(string t){ cout<<"[Led] "<<t<<endl; } };
class Cluster {                           // keeps members together
    Account a; Ledger l;
public:
    void process() {
        a.run("debit Rs.2500");           // stays in-process
        l.run("record debit entry");      // no marshalling
        cout << "Done locally." << endl;
    }
};
int main(){ Cluster b; b.process(); return 0; }
```

Output

OUTPUT

```
[Acc] debit Rs.2500
[Led] record debit entry
Done locally.
```

CONCLUSION

A good clustering factor keeps the objects that talk most often inside one boundary, so marshalling disappears and both speed and reliability improve.

02 QUESTION TWO · PERSISTENCE

Explain the meaning of persistence in COM / DCOM.

4 A4 PAGES
C++ PROGRAM ON PAGE 4

MEANING OF PERSISTENCE IN COM / DCOM

Persistence is the ability of an object to save its **state** in a permanent store and restore it later, even after the program has closed or the machine been switched off. The *state* is simply the values held in its data members â€" for a bank object, the account number, the holder's name and the balance. RAM is **volatile**, so the moment the process ends everything it held is wiped out. Persistence is what lets an object survive that moment.

A COM object writes itself into a **stream** or a **storage** and reads itself back later. Turning a live object into bytes is **serialisation**; rebuilding it is **deserialisation**. A video game shows this clearly: while you play, your score and level stay only in memory, so a power cut loses everything. *Save Game* writes those values to a disk file and *Load Game* reads them back â€" which is exactly what a persistent COM object does.

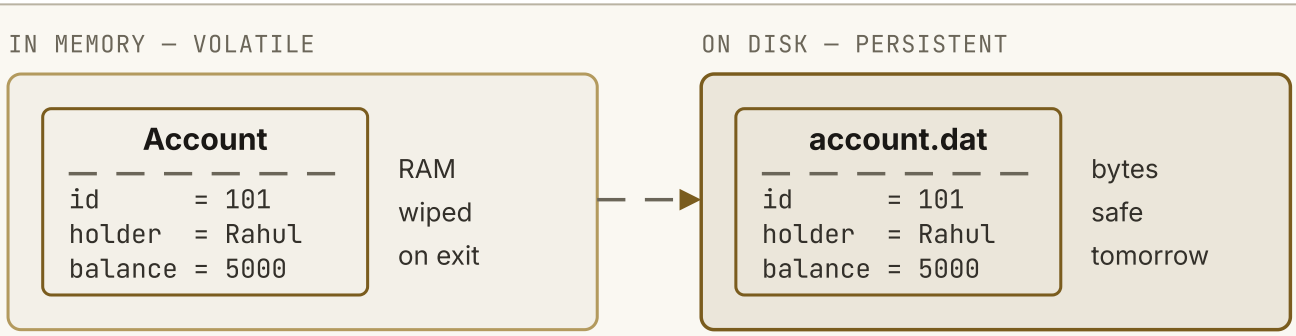


FIG. 1 — THE STATE OF ONE OBJECT, SEEN TWICE. ON THE LEFT IT LIVES ONLY IN VOLATILE RAM AND DIES WITH THE PROCESS; ON THE RIGHT THE SAME VALUES HAVE BEEN WRITTEN INTO A FILE, SO THEY CAN BE READ BACK TOMORROW AND THE OBJECT REBUILT AS IT WAS.

WHY PERSISTENCE IS NEEDED

No program is expected to run for ever â€" the user closes it, the machine restarts, and sometimes a crash occurs. Without persistence every useful object would vanish on each shutdown. Persistence lets a document, a customer record, a user's preferences or a partly-finished transaction live safely between runs. It also allows data to be **shared**, because one program can save an object and another open it later, and it lets a system **recover** after a failure, because the last saved state is still on the disk.

HOW COM SUPPORTS PERSISTENCE

In **data-centric persistence** the component saves only its plain data and must know a fixed file layout. In **object-centric persistence** the object saves its own state through standard interfaces, so COM supports it with `IPersistStream` and `IPersistStorage`.

The store comes from **Structured Storage**, a small file system living inside one file. Inside it, **storages** work like folders and **streams** like files. A **moniker** is a name that finds the object again, so a saved object is linked rather than merely copied.

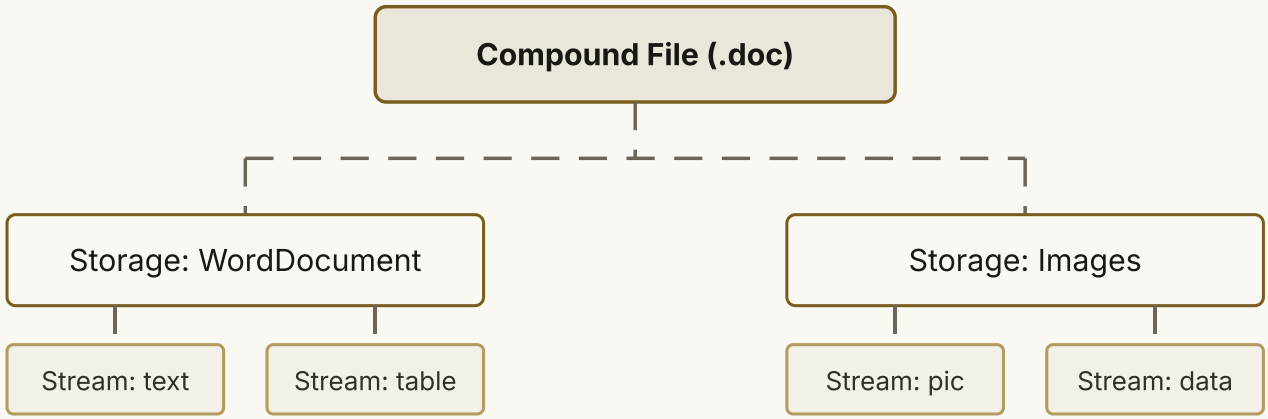


FIG. 2 — STRUCTURED STORAGE. A COMPOUND FILE BEHAVES LIKE A SMALL FILE SYSTEM: STORAGES ACT AS FOLDERS AND STREAMS AS FILES, ALL INSIDE A SINGLE ORDINARY FILE.

PERSISTENCE IN DCOM

In DCOM the object lives on a remote machine, so persistence also decides **where** the work resumes. The server keeps the object alive only while a client is using it; when the client disconnects the object is unloaded and its state is written back to the server's disk. The next time a client asks for that object by name, the server **activates** it again and rebuilds it from the saved state. Nothing is lost, and the client never has to know that the process died in between.

TERM	MEANING
Storage	The place where bytes are kept
Serialisation	Converting an object into those bytes
Persistence	The promise that the state survives
Moniker	A name that finds the object again

A student result project shows this: the object is saved, the program closes, and reopening it activates the same object with the marks intact.

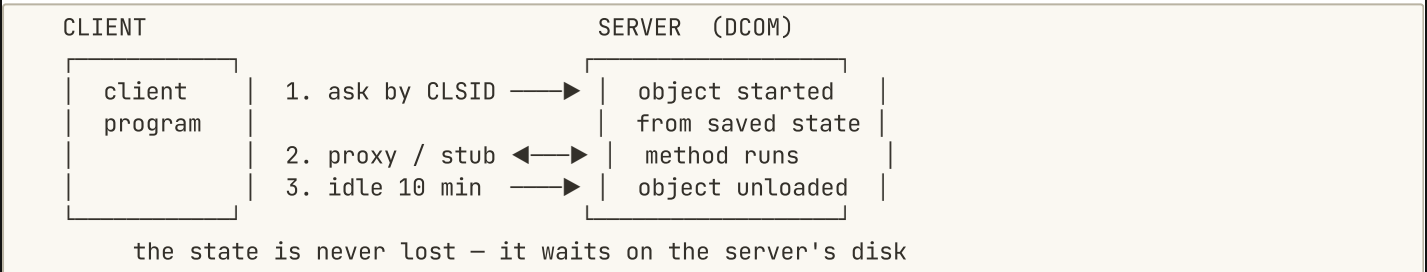


FIG. 3 — PERSISTENCE IN DCOM, DRAWN AS A CONVERSATION. THE CLIENT NEVER KEEPS THE OBJECT ALIVE; IT ASKS FOR IT BY NAME, THE SERVER REBUILDS IT FROM ITS SAVED STATE, AND WHEN THE OBJECT FALLS IDLE IT IS UNLOADED AGAIN — ITS STATE KEPT SAFELY ON DISK.

C++ PROGRAM

PERSIST_DEMO.CPP

```cpp
// persist_demo.cpp - save state, restore it later
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
class Account {                       // a persistent COM-style object
    int id; string holder; double bal;
public:
    Account(int i,string h,double b):id(i),holder(h),bal(b){}
    void save() {                      // serialise into a stream
        ofstream f("account.dat");
        f << id << "|" << holder << "|" << bal;
    }
    void load() {                      // deserialise from the same file
        ifstream f("account.dat");
        f >> id >> holder >> bal;
    }
    void show(){ cout << id << " " << holder << " " << bal << endl; }
};
int main(){
    Account a(101,"Rahul",5000); a.save(); cout << "saved" << endl;
    Account b(0,"",0);          b.load(); b.show();   // rebuilt
    return 0;
}
```

Output

OUTPUT

```
saved
101 Rahul 5000
```

CONCLUSION

Persistence lets a COM object outlive the process that made it, so its state is available again on any later run.